SOFTWARE ENGINEERING MINI PROJECT DELIVERABLE 1

Software Requirement Specification (SRS)

Team 11

Ianna Elizabeth Reni	PES2UG24CS636
Pranav Reddy Melachervu	PES2UG24CS674
Vikas N	PES2UG24CS668
Pratheek P Shettigar	PES2UG24CS650

Software Requirements Specification (SRS)

Project: Bank Management System
Version: 1.0
Authors: Ianna Elizabeth Reni, Pranav Reddy Melachervu, Vikas N, Pratheek P Shettigar
Date: 11-09-2026
Status: Draft

Revision history

Version	Date	Author	Change summary	Approval
1.0	11-09-2026	Team 11	Initial SRS draft	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

The Bank Management System has been designed to provide a computerized environment for the processing and managing of basic banking processes. This system has been developed to allow the customers and the administrators to perform their basic banking tasks such as account management, deposit, withdrawal, account checking, money transfer, and transaction history.

The system is going to be designed using C/C++ and will provide a user-friendly interface for carrying out transactions in the bank. The system will save the account details of the customers and transaction details while at the same time providing validation for any transactions carried out.

The Software Requirements Specification (SRS) document describes the functional and non-functional requirements, behavior of the system, interfaces, constraints, security requirements, and the acceptance criteria of the Bank Management System. This document is used as a guiding framework for the development, testing, and evaluation of the system.

1.2 Scope

The Bank Management System is a C/C++ application designed for simulating and managing the usual banking tasks in a software environment. This application stores all information related to the customers' accounts and helps the authorized user perform basic banking transactions.

The system includes the following main features:

- Customer accounts creation and management.
- Customer login with proper credentials.
- Displaying customer and account information.
- Transaction with the deposit to the account.
- Transaction with the withdrawal from the account.
- Checking the current account balance.
- Fund transfer between eligible accounts.
- Checking the transaction history.
- Customer information update.
- Changing account login credentials.
- Closing of an account if needed.

- Customer accounts administration.
- Validating banking transactions to avoid any invalid transactions.

All the data about the accounts and transactions is stored locally and accessed through the menu-based interface.

This application will be used solely as a learning tool for managing a bank, and it is not meant to be a connection to an actual bank. Services like UPI payments, ATM hardware support, credit and debit cards processing, loan processing, online banking services, inter-bank transactions, and external banking network connections are beyond the scope of this project.

1.3 Audience

The target audience for this Software Requirements Specification is described as follows:

- Project Developers: In order to understand the functionality, behavior, interfaces, constraints, and implementation expectations of the Bank Management System.

- Project Testers: In order to create test cases and determine whether the implemented system meets the requirements set out for functionality, non-functionality, and security.

- Project Team Members: In order to maintain an understanding of the scope, features, and requirements of the system throughout the process of its development.

- Project Guide and Faculty Evaluators: In order to assess the requirements, approach taken in design, scope of the implementation, and general quality of the project.

- End Users: In order to understand the main functions provided by the Bank Management System.

1.4 Definition

- BMS: Bank Management System: software application designed to manage core banking transactions.

- Customer: Registered user with a bank account in the system.
- Administrator: Authorized user accountable for the management of customer accounts and overall system operations.
- Bank Account: Database record holding customer data, account information and balance.
- Account Number: Unique account identifier assigned to each bank account.
- Authentication: Verification of user's identity based on valid credentials, such as password or PIN.
- Deposit: Banking transaction that increases the balance of the account by a certain amount.
- Withdrawal: Banking transaction that reduces the balance of the account by a certain amount subject to certain validations.
- Balance Inquiry: Retrieval and display of current account balance.
- Fund Transfer: Process of transferring a certain amount of money between two eligible accounts within the system.
- Transaction: Recorded banking transaction, e.g. deposit, withdrawal or fund transfer.
- Transaction History: List of previously completed transactions of the account.
- Account Closure: Process of closing of an existing bank account in accordance with system rules.
- Local Data Storage: The means of storing account and transaction records in the local system by the application.
- SRS: Software Requirements Specification: document outlining system requirements and its expected behaviour.

2. Overall description

2.1 Product Perspective

The Bank Management System is a standalone application based on C/C++ which has been developed in order to simulate and manage basic banking operations; it offers a structured software environment enabling customers and authorised administrators to carry out banking-related activities using a user-friendly interface.

Customer account information and transaction records are stored locally by the system, and the application functions as the main component for carrying out banking operations and for verifying transactions in accordance with the predefined system rules.

The major interaction between the users and the system can be represented as follows:

Customer → Authentication → Bank Management System → Banking Operations → Data Storage

Administrator → Authentication → Bank Management System → Account/System Management → Data Storage

The Bank Management System is intended to be an independent academic application and makes no use of external banking networks or actual financial institutions. All operations relating to accounts and transactions take place within the system's controlled environment.

2.2 Major product functions (detailed)

The Bank Management System will include the following main functionalities:

- User Authentication
    - The system will authenticate users (both customers and administrators) using appropriate credentials before allowing access to secured functionalities.
- Customer Account Management
    - The system will provide the functionality for creating, viewing, modifying, and closing the customer accounts according to the system rules.
- Customer Information Management
    - The system will store relevant information of the customer, such as name, contact details, address, and any other information related to accounts.

- Deposit Management
    - The system must provide an option for a customer authorized to make a deposit of a certain amount into his/her account and update the account balance accordingly.
- Withdrawal Management
    - The system must provide an option for a customer authorized to make a withdrawal from his/her account, provided there is sufficient account balance.
- Checking the Account Balance
    - The system must provide an option for a customer to check the account balance.
- Funds Transfer
    - The system must provide an option for a customer to transfer money from one account to another, provided there is validation.
- Transaction History
    - The system will allow customers to check their transaction history associated with their accounts.
- Credential Management
    - The system will allow the customer with the required authorization to change their credentials as per security requirements.
- Administrative Management
    - The system will allow the administrators with authorization to manage customer accounts and access relevant records of the system.
- Input Validation and Error Messages
    - The system will validate input and will return the relevant message for any invalid input or operation not allowed.
- Data Persistence
    - The system will save and retrieve data for account and transaction from the local data storage.

2.3 User Roles and Characteristics

The Bank Management System is designed to be used by customers and authorized administrators. Each user role has different responsibilities and access privileges within the system.

2.3.1 Customer

The customer is a registered user who owns a bank account and uses the system to perform basic banking operations.

The customer can:

- Authenticate using the required credentials.
- View account and personal information.
- Check the current account balance.
- Deposit money.
- Withdraw money.
- Transfer funds to eligible accounts.
- View transaction history.
- Update permitted personal information.
- Change account authentication credentials.
- Request account closure when applicable.

The customer is expected to have basic knowledge of banking operations and should be able to use the menu-driven interface without requiring technical knowledge.

2.3.2 Administrator

The administrator is an authorized user responsible for managing customer accounts and system-level banking records.

The administrator can:

- Authenticate using administrator credentials.
- View customer account information.
- Create or manage customer accounts where permitted.
- Update or verify customer information.
- Search for customer accounts.
- Manage account status.
- View relevant transaction records.
- Perform administrative operations according to system permissions.

The administrator is expected to have a basic understanding of banking procedures and system operations.

2.3.3 System User Characteristics

The system is intended for users with basic computer literacy. The interface should provide clear menus, understandable instructions, appropriate validation messages, and error messages so that users can complete operations with minimal difficulty.

Different user roles shall have access only to the operations permitted for their role.

2.4 Operating Environment

The Bank Management System is supposed to work as an autonomous program on personal computers and portable computers without any connection to other banking computer networks and financial institutions.

2.4.1 Hardware Environment

The system can run on common PC with the following minimum configuration:

- Processor: Dual-core or higher.
- RAM: 4 GB or more.
- Free space on the hard drive: not less than 500 MB for the program and its data files.
- Input devices: keyboard and mouse.
- Output device: computer monitor.

2.4.2 Software Environment

C/C++ programming language and the following software are needed for creation and operation of the system:

- Operating system: Windows or Linux.
- C/C++ compiler supporting at least C++17 standard.
- Standard C++ library of the program.
- File system for storing account and transaction data.

2.4.3 Application Environment

The application has a menu-based interface allowing users and administrators to use functions appropriate for their role.

Accounts and transactions information is stored locally using file data storage. The program loads all needed data at the start and saves changes to data files after finishing banking operations.

The system was developed for use in academic environments and doesn't require internet connection for its main functionality.

2.5 Constraints

The design and execution of the Bank Management System is constrained by the following constraints:

1. Programming Language Constraint: The system should be developed using C/C++ as per the specifications of the project.

2. Type of Application Constraint: The application should work as a standalone application without depending on any banking systems or APIs from any financial institutions.

3. Data Storage Constraint: Data related to accounts and transactions should be stored using local data storage (such as file); the basic functionality of the system does not need a cloud-based database.

4. Authentication Constraint: There should be proper authentication in order to access the operations related to customers and administrative activities, and only the operations assigned to the user's role should be performed.

5.Transaction Validation Constraint: Any operation involving any banking functionality, such as withdrawals or transfers of funds, should validate the transactions before being finalized. Transactions with any incorrect inputs such as wrong amounts, insufficient balances, wrong account IDs, or other invalid data should be refused.

6.Data Integrity Constraint: The system should maintain consistent account and transaction information while executing any banking operations.

7.Platform Constraint: The system should be built in a way to be executed on any common platform such as Windows or Linux that supports C++ programming language.

8.Scope Constraint: The project should serve as an academic project and should demonstrate only basic banking operations. Actual banking services like hardware integration with ATMs, UPI, card networks, inter-bank transfer facilities, loans, or any connections with the bank are not in the scope of this project.

9.Security Constraint: The system should provide some basic security features appropriate to academic use, which would include authentication and access control to protect sensitive information in a C/C++ local application environment.

10.Resource Constraint: The system should be designed in such a way as to work on a general computer with no need for any banking-specific hardware or software.

3. External interface requirements

3.1 User interfaces

The interface of the system should be easy and user friendly for both bank customers and employees. This interface will enable the user to log in, view account information, deposit money, withdraw money, transfer money, and view transaction history.

3.2 Hardware interfaces

- Computer/terminal used by bank employees.
- Keyboard and mouse for data entry.
- Display/monitor for showing account and transaction information.
- Printer for printing receipts and transaction statements.
- ATM card reader and cash dispenser if ATM functionality is included.
- Status LEDs and operator panel

3.3 Software interfaces

- Database Management System for storing customer, account, and transaction information.
- Banking application/API for processing banking operations.
- Authentication module for validating customer and employee credentials.

3.4 Communications

- Communication shall use TLS 1.2 or higher where network communication is required.
- The system shall handle temporary network failures without corrupting transaction data.

4. System features (detailed)

4.1 Authentication

Description: Authenticate the cardholder using EMV card and PIN; support chip & contactless where applicable.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
BMS-F-001	The system shall validate card authenticity (EMV) and verify PIN via bank core.	Functional	High	Security / Banking	AC-BMS-F-001: Valid EMV card and correct PIN leads to authenticated session. Test: TC-Auth-01	Requires EMV reader and Core API
BMS-F-002	The system shall lock the card after 3 consecutive failed PIN attempts and record an audit event.	Functional	High	Security	AC-BMS-F-002: After 3 failures, further attempts are blocked and message displayed. Test: TC-Auth-03	Card must be flagged in Core
BMS-F-003	The system shall allow an authenticated customer to change their login credentials (password/PIN) after re-verifying current credentials.	Functional	Medium	Security	AC-BMS-F-003: Credential change succeeds only after correct current PIN is entered; new credentials meet complexity rules. Test: TC-Auth-04	Allows users after creation of an account to change login details.

BMS-F-004	The system shall terminate the session and require re-authentication after a period of inactivity or on explicit logout.	Functional	Medium	Security	AC-BMS-F-004: Session ends after timeout/logout; further actions require re-login. Test: TC-Auth-05	Protects users from old session tokens.
BMS-F-005	The system shall allow users to create new accounts	Functional	High	Security	AC-BMS-F-005:Cust Customer should provide valid information about themselves, reed KYC for more details.: TC-Auth-05	Allows users to create multiple accounts under their name.

4.2 Cash Withdrawal

Description: Dispense cash in validated denominations and ensure ledger consistency with bank core.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
BMS-F-010	The system shall verify sufficient available balance and daily limit before dispensing cash.	Functional	High	Business	AC-BMS-F-010: Dispense occurs only if balance >= amount and within limits. Test: TC-WD-01	Core API check required

BMS-F-011	The system shall dispense requested amount using available denominations and update cash cassette counts.	Functional	High	Operations	AC-BMS-F-011: Dispensed amount equals requested; cassette counts decremented. Test: TC-WD-02	Hardware interaction with dispenser

4.3 Cash/Check Deposit

Description: The system should enable the user to enter the amount of money that he or she wishes to deposit using the available method. The system should validate the entered amount, update the balance of the customer's account upon the deposit, and create a record of the transaction. IDs: BMS-F-020..BMS-F-025.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
BMS-F-020	The system shall allow the user to enter the amount to be deposited.	Functional	High	Customer / Banking	AC-BMS-F-020:Valid deposit amount is accepted. Test: TC-DEP-01	Input validation required
BMS-F-021	The system shall accept cash or check deposits through the supported deposit mechanism. .	Functional	High	Banking	AC-BMS-F-011: Dispensed amount equals requested; cassette counts decremented. Test: TC-WD-02	Hardware interaction with dispenser
BMS-F-023	The system shall update the customer's account balance after a	Functional	High	Banking	AC-BMS-F-023: Account balance reflects the	Requires Core API

	successful cash deposit.				deposited amount. Test: TC-DEP-04	
BMS-F-024	The system shall generate a transaction record for every successful deposit.	Functional	High	Banking/Audit	AC-BMS-F-024: Deposit transaction is recorded with date, time, amount, and account. Test: TC-DEP-05	Database required

4.4 Funds Transfer

Description: The system must enable transferring money between eligible accounts. Prior to effecting the transfer, the system must validate whether there is adequate balance and set limits for the transfer. Post a successful transfer, the system must update the balances of the source and destination accounts. IDs: BMS-F-030..BMS-F-032.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
BMS-F-030	The system shall allow users to transfer funds between eligible accounts	Functional	High	Customer / Banking	AC-BMS-F-030: Valid transfer is successfully processed. Test: TC-TR-01	Core API required

BMS-F-031	The system shall verify sufficient funds and configured transfer limits before processing a transfer.	Functional	High	Banking /Security	AC-BMS-F-031: Transfer is rejected when balance or limit is insufficient. Test: TC-TR-02	Core API check required
BMS-F-032	The system shall update the source and destination account balances after a successful transfer and generate a confirmation.	Functional	High	Banking / Customer	AC-BMS-F-032: Both balances are updated correctly and confirmation is displayed. Test: TC-TR-03	Transaction must be atomic

4.5 Maintenance & Monitoring

Description: The system shall present a technician menu for the diagnosis and maintenance of the system. The system shall monitor connected hardware, log system errors and events, display system status to authorized bank operators, send alarms on failures, and conduct system tests. IDs: BMS-F-040..BMS-F-045

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
BMS-F-040	The system shall provide an authorized technician menu for performing system diagnostics.	Functional	High	Technician	AC-BMS-F-040: Authorized technician can access diagnostic functions. Test: TC-MNT-01	Admin authentication required

BMS-F-041	The system shall record system errors and important operational events in logs.	Functional	High	Operations / Security	AC-BMS-F-041: Errors and events are recorded with timestamps. Test: TC-MNT-03	Logging module required
BMS-F-042	The system shall provide remote monitoring information to authorized bank operators.	Functional	Medium	Bank Operator	AC-BMS-F-042: Authorized operator can view system status remotely. Test: TC-MNT-04	Monitoring API required
BMS-F-043	The system shall notify authorized personnel when a critical system or hardware failure occurs.	Functional	High	Operations	AC-BMS-F-043: Alert is generated for critical failures. Test: TC-MNT-05	Alerting service required

5. Non-functional requirements (detailed)

Req ID	Requirement	Category	Priority	Acceptance criteria / Test case ref
BMS-NF-001	Any menu operation (login, deposit, withdrawal, balance inquiry, transfer, history lookup) shall return a result within 2 seconds on a standard desktop/laptop.	Performance	High	AC-NF-001: 95% of operations complete in ≤2s during local test runs. Test: TC-Perf-01

BMS-NF-002	The system shall not lose or corrupt account or transaction data if the application is interrupted mid-transaction (e.g., abrupt exit, power loss).	Reliability	High	AC-NF-002: Forced interruption during a transaction leaves the data file in its pre- or post-transaction state, never partially written. Test: TC-Rel-01
BMS-NF-003	The menu-driven interface shall be understandable by a first-time user without a manual, using numbered options and clear prompts/error messages.	Usability	Medium	AC-NF-003: A first-time test user completes login, deposit, and balance inquiry without external help. Test: TC-UX-01
BMS-NF-004	The system shall be organized into independent modules (authentication, account management, transactions, reporting) so a change to one does not require modifying the others.	Maintainability	Medium	AC-NF-004: Code review confirms module separation; a sample feature change touches only its own module. Test: Code review checklist
BMS-NF-005	The system shall compile and run on standard Windows and Linux environments using a standard C/C++ compiler, with no proprietary or specialized hardware dependency.	Portability	Medium	AC-NF-005: Successful build and run demonstrated on both a Windows and a Linux machine from the same source. Test: TC-Port-01

5.1. Security

5.1.1 Security Objectives

This project has the following security objectives, scoped to a locally-run, single-application system:

- Confidentiality: Prevent customer passwords/PINs and account data from being exposed in plaintext, whether on-screen, in the data file, or in logs.
- Integrity: Prevent unauthorized or accidental modification of account balances and transaction records, and ensure every balance change is traceable to a specific transaction.

- Accountability: Ensure every login attempt and every transaction (successful or failed) is logged so activity can be reviewed after the fact.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
BMS-SR-001	The system shall store customer login credentials (password/PIN) in hashed form; plaintext credentials shall never be written to the data file.	Security	High	AC-SR-001: Inspecting the data file shows no plaintext password/PIN values. Test: TC-Sec-01
BMS-SR-002	The system shall mask password/PIN entry on-screen (e.g., display asterisks) during login.	Security	High	AC-SR-002: Characters typed at the login prompt are not echoed in plaintext. Test: TC-Sec-02
BMS-SR-003	The system shall lock a customer account after 3 consecutive failed login attempts in a session and require administrator action to unlock it.	Security	High	AC-SR-003: A 4th consecutive failed attempt is rejected with a lockout message; login only succeeds again after admin reset. Test: TC-Sec-03
BMS-SR-004	The system shall validate all transaction amounts (deposit, withdrawal, transfer) to reject negative, zero, or non-numeric input before touching the stored balance.	Security	High	AC-SR-004: Invalid amounts are rejected with an error and the stored balance is unchanged. Test: TC-Sec-04
BMS-SR-005	The system shall record every login attempt and every transaction (successful or failed) with a timestamp, account ID, and outcome in a local audit log.	Security	High	AC-SR-005: Audit log contains a timestamped entry for each attempted login and transaction generated during a test session. Test: TC-Sec-05

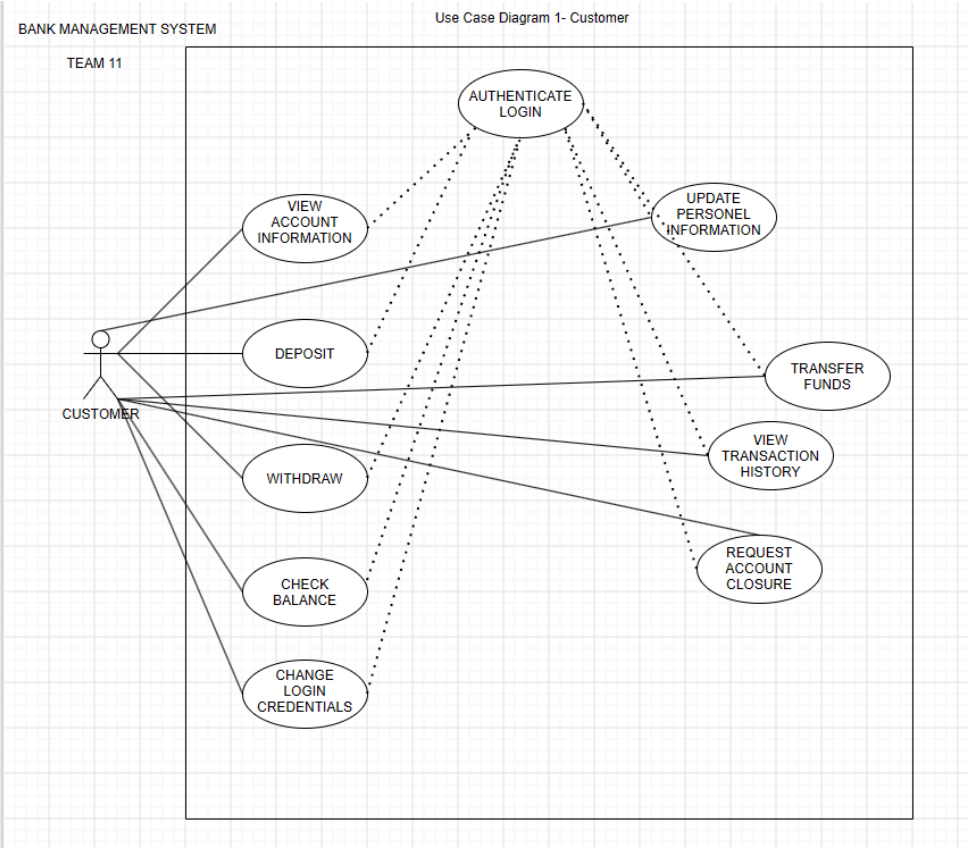
6. Quality attributes & Acceptance tests

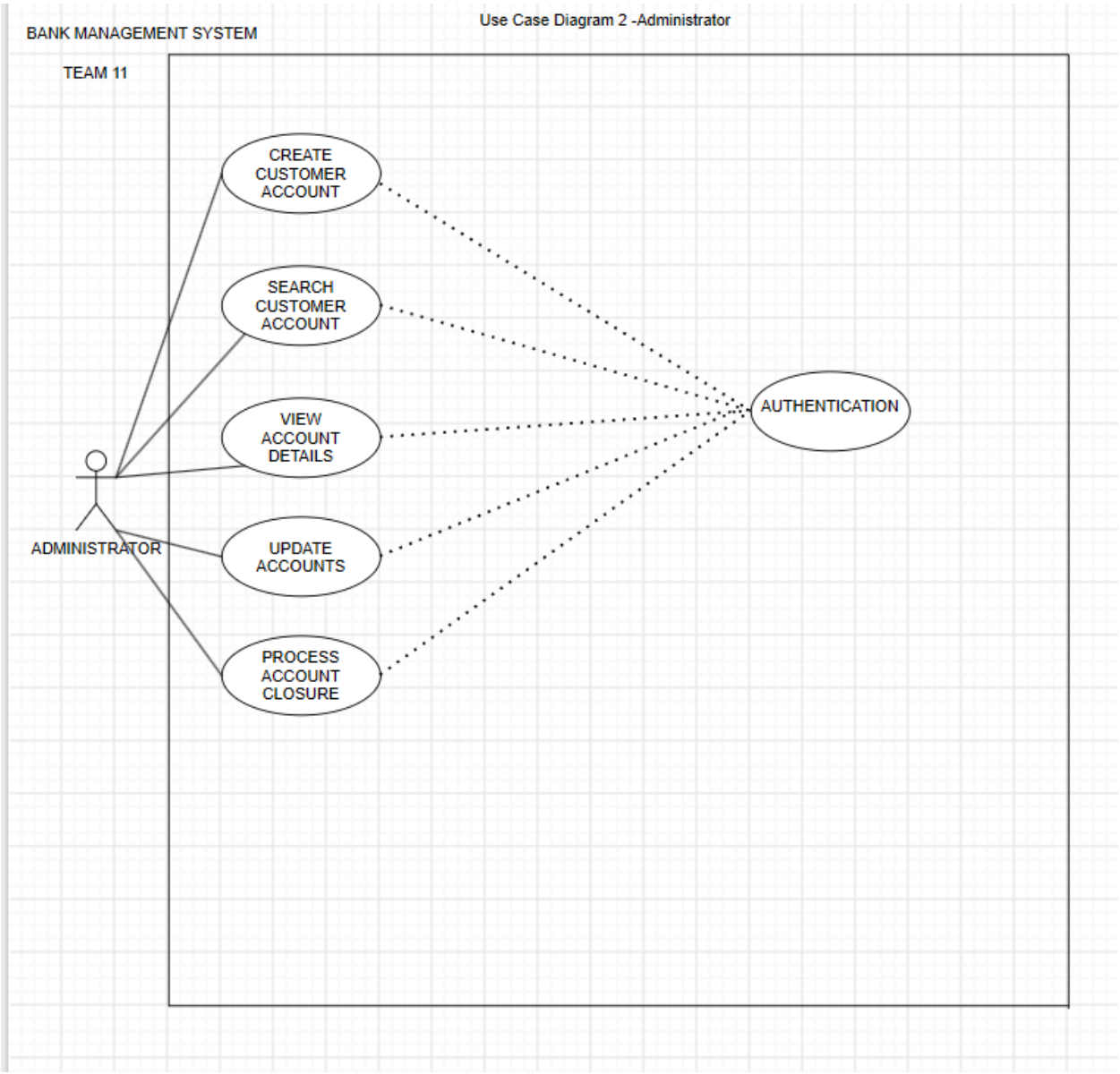
Exit criteria for acceptance: All high-priority functional requirements from Section 4 shall be implemented and verified. The external interfaces and system operations described in Section 3 shall function correctly. All high-priority non-functional and security requirements from Section 5 shall meet their acceptance criteria. No critical authentication, transaction, data integrity, or security failures shall remain unresolved. The RTM shall contain the required test case references and all required tests shall be successfully completed.

Acceptance test suites: Authentication, Withdrawal, Deposit, Funds Transfer, Maintenance and Monitoring, Performance, Reliability, Usability, Portability, and Security tests shall be performed. The tests shall verify the system features, transaction processing, user interaction, data integrity, response time, platform compatibility, authentication, transaction validation, and audit logging.

7. System models and diagrams

7.1 UML Use-Case diagram




Use Case Diagram 2 -Administrator
BANK MANAGEMENT SYSTEM
TEAM 11
CREATE CUSTOMER ACCOUNT
SEARCH CUSTOMER ACCOUNT
VIEW ACCOUNT DETAILS
UPDATE ACCOUNTS
PROCESS ACCOUNT CLOSURE
AUTHENTICATION
ADMINISTRATOR

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement Short	Section Ref / Design Spec	Module	Test Case(s)	Status (N/P/A)	Comments
BMS-F-001	Validate card authenticity and PIN	4.1	Authentication	TC-Auth-01	N	Requires EMV reader and Core API
BMS-F-002	Lock card after 3 failed PIN attempts	4.1	Authentication	TC-Auth-03	N	Card must be flagged in Core
BMS-F-003	Change login credentials after re-verification	4.1	Authentication	TC-Auth-04	N	Requires current credential verification
BMS-F-004	Terminate session after inactivity or logout	4.1	Authentication	TC-Auth-05	N	Protects users from old session tokens
BMS-F-005	Allow users to create new accounts	4.1	Authentication	TC-Auth-05	N	Valid customer information required
BMS-F-010	Verify balance and daily limit	4.2	Cash Withdrawal	TC-WD-01	N	Core API check required
BMS-F-011	Dispense requested cash and update cassette count	4.2	Cash Withdrawal	TC-WD-02	N	Hardware interaction with dispenser
BMS-F-020	Enter deposit amount	4.3	Cash/Check Deposit	TC-DEP-01	N	Input validation required

BMS-F-021	Accept cash or check deposit	4.3	Cash/Check Deposit	TC-WD-02	N	Source Section 4.3 provides TC-WD-02; verify test reference
BMS-F-023	Update balance after successful deposit	4.3	Cash/Check Deposit	TC-DEP-04	N	Requires Core API
BMS-F-024	Generate deposit transaction record	4.3	Cash/Check Deposit	TC-DEP-05	N	Database required
BMS-F-030	Transfer funds between eligible accounts	4.4	Funds Transfer	TC-TR-01	N	Core API required
BMS-F-031	Verify funds and transfer limits	4.4	Funds Transfer	TC-TR-02	N	Core API check required
BMS-F-032	Update source and destination balances	4.4	Funds Transfer	TC-TR-03	N	Transaction must be atomic
BMS-F-040	Provide authorized technician diagnostics	4.5	Maintenance & Monitoring	TC-MNT-01	N	Admin authentication required
BMS-F-041	Record system errors and operational events	4.5	Maintenance & Monitoring	TC-MNT-03	N	Logging module required
BMS-F-042	Provide remote monitoring information	4.5	Maintenance & Monitoring	TC-MNT-04	N	Monitoring API required

BMS-F-043	Notify personnel of critical failures	4.5	Maintenance & Monitoring	TC-MNT-05	N	Alerting service required
BMS-NF-001	Menu operation response within 2 seconds	5	Performance	TC-Perf-01	N	Performance requirement
BMS-NF-002	Prevent data loss or corruption on interruption	5	Reliability	TC-Rel-01	N	Reliability requirement
BMS-NF-003	Provide understandable menu-driven interface	5	Usability	TC-UX-01	N	Usability requirement
BMS-NF-004	Maintain independent system modules	5	Maintainability	Code Review Checklist	N	Maintainability requirement
BMS-NF-005	Run on standard Windows and Linux environments	5	Portability	TC-Port-01	N	Portability requirement
BMS-SR-001	Store credentials in hashed form	5.1	Security	TC-Sec-01	N	Confidentiality
BMS-SR-002	Mask password/PIN entry	5.1	Security	TC-Sec-02	N	Confidentiality
BMS-SR-003	Lock account after 3 failed login attempts	5.1	Security	TC-Sec-03	N	Access control

BMS-SR-004	Validate transaction amounts	5.1	Security	TC-Sec-04	N	Data integrity
BMS-SR-005	Record login and transaction audit events	5.1	Security / Audit Log	TC-Sec-05	N	Accountability